```
if(modelNamespace + ".SalesOrder.SalesOrderEvent" == actualEvent.eventType) {
…
}
```

System Embedded Functions

There are some predefined system functions that you can invoke directly in the script.

System embedded functions are divided into two groups:

- Event to Action Internal functions: are used within SAP Business Network Global Track and Trace.
- Event to Action External functions: interact with third-party systems in their operations.

Event to Action Internal Functions

Event to Action Internal functions are used within SAP Business Network Global Track and Trace. They include:

`toTimestamp`

Function `toTimestamp` converts a timestamp in string format into a timestamp value.

```
/**
 * Convert a timestamp in string format into TimestampValue.
 *
 * @param timeString - A timestamp in string format. Example: "2013-06-25T16:22:52.966Z".
 * @returns TimestampValue
 */
toTimestamp(timeString)
```

`toDate`

Function `toDate` converts a date in string format into a date value.

```
/**
 * Convert a date in string format into DateValue.
 *
 * @param dateString - A date in string format. Example: "2019-05-04".
 * @returns DateValue
 */
toDate(dateString)
```

`toUUID`

Function `toUUID` converts a UUID in string format into a UUID value.

```
/**
 * Convert a UUID in string format into UUIDValue.
```

```
 *
 * @param uuidString - A UUID string. Example: "00000000-0000-0000-0000-000000000000".
 * @returns UUIDValue
 */
toUUID(uuidString)
```

toObject

Function `toObject` converts a JSON string into an object value.

```
/**
 * Convert a JSON string into ObjectValue.
 *
 * @param jsonString - A JSON string.
 * @returns ObjectValue
 */
toObject(jsonString)
```

toString

Function `toString` converts a timestamp value, a date value, or an object value into a string. The resulting string for timestamps and dates follows a set pattern, as shown in the examples below.

```
/**
* Convert a timestamp value, a date value, or an object value into a string.
*
* @returns String
*/
date.toString()
time.toString()
object.toString()
```

Examples:

For dates:

```
var date = toDate("2022-03-19");
var string = date.toString(); // The value of string is: 2022-03-19
```

For timestamps:

```
var time = toTimestamp("2022-03-19T16:22:52.966Z");
var string = time.toString(); // The value of string is: 2022-03-19T16:22:52.966Z
```

For objects:

```
var object = toObject("{}");
var string = object.toString(); // The value of string is: {}
```

updateTPProperties

Function `updateTPProperties` updates properties of a tracked process.

```
/**
* Update properties of a tracked process.
*
* @param trackedProcess - A trackedProcess object.
* @param properties - An object contains the properties that need to be updated in the tracked proc
* @returns null
*/
updateTPProperties(trackedProcess, properties)
```

For example:

```
var obj = toObject("{}");
var a = toObject("{\n" +
                 "               \"batchNumber\": null,\n" +
                 "               \"deliveryItem\": \"000020\",\n" +
                 "               \"deliveryItemRef\": \"102\",\n" +
                 "             }");

var b = toObject("{\n" +
                 "               \"batchNumber\": null,\n" +
                 "               \"deliveryItem\": \"000030\",\n" +
                 "               \"deliveryItemRef\": \"102\",\n" +
                 "             }");
obj.DeliveryItem = [a, b];
updateTPProperties(trackedProcess, obj);
```

updatePlan

After you change the timestamp of the planned event in the event-to-action script, you can use the function `updatePlan` to recalculate the planned event status and the tracked process status, and to reset the overdue detection.

```
/**
* Recalculate planned event status and tracked process status, reset overdue detection. Changes are
*
* @optional param plannedEventIdArray - An array of planned event IDs that need to be recalculated.
* @returns null
*/
trackedProcess.updatePlan([id1, id2, ...]) - We highly recommend you provide a parameter for better
```

For example:

```
var plannedEvents = trackedProcess.plannedEvents;
plannedEvents[0].plannedBusinessTimestamp = toTimestamp("2021-06-16T11:00:00.213+01:00");
plannedEvents[0].plannedBizTsEarliest = toTimestamp("2021-06-16T10:00:00.213+01:00");
plannedEvents[0].plannedBizTsLatest = toTimestamp("2021-06-16T12:00:00.213+01:00");
var id = plannedEvents[0].id;
trackedProcess.updatePlan([id]);
```

getLocationByAltKey

Function `getLocationByAltKey` gets the location master data from the Manage Locations app through the location alternative key. The use of this function may affect event processing perfomance.

```
/**
* Get the location master data
*
* @param locationAltkey - A string
* @returns ObjectValue of location data
*/
getLocationByAltKey(locationAltkey)
```

The function returns an object value with the following properties:

Response Properties

• StreetName • HouseNumberSupplementText • HouseNumber • RegionCode • CountryName • IataAirportCode • AddressTimeZone • PostalCode • Latitude • Longitude	• CityName • EmailAddress • Unlocode • SourceUniversalObjectId • LocationDescription • Number • RegionName • NumberExtension • CountryCode • LocationTypeCode	• LocationTypeDescription • SourceSystem • SourceSystemType • ExternalId • ObjectTypeCode • ObjectTypeDescription • LocationTypeCode • LocationTypeDescription

Examples:

Using the function:

```
var loc = getLocationByAltKey("xri://sap.com/id:LBN#10013165:01:Location:LogisticLocation:02");
```

Response:

```
{
        "StreetName": null,
        "HouseNumber": null,
        "RegionCode": null,
        "CountryName": null,
        "ObjectTypeCode": "LogisticLocation",
        "PostalCode": null,
        "Latitude": null,
        "SourceSystemType": "SCM",
        "LocationDescription": "alex test logistic location",
        "Number": null,
```

```
        "RegionName": null,
        "LocationTypeCode": "1001",
        "CountryCode": null,
        "HouseNumberSupplementText": null,
        "ObjectTypeDescription": "Logistic Location",
        "IataAirportCode": "",
        "LocationTypeDescription": "Production Plant",
        "ExternalId": "001",
        "AddressTimeZone": null,
        "CityName": null,
        "Longitude": null,
        "EmailAddress": null,
        "Unlocode": "",
        "SourceUniversalObjectId": "xri://sap.com/id:LBN#10013165:LOGISTICLOCATION:Location:Logist
        "NumberExtension": null,
        "SourceSystem": "LOGISTICLOCATION"
    }
```


Using a response property:

```
loc.StreetName
```

getEventTypeInfo

Function `getEventTypeInfo` gets the name, description, and fields of a given event type. This function responds in the following ways when dealing with varied situations:

- If a Java locale string is provided, it will return the description in the specified language.
- If there is no text in the specified language, it will default to returning the text in English.
- If the specified event type doesn't exist, it will return an empty string.

```
/**
 * Get the name, description, and fields of a given event type.
 *
 * @param eventTypeFullName - A string.
 * @param languageCode - A Java locale string.
 * @returns ObjectValue of the event type info
 */
getEventTypeInfo(eventTypeFullName, languageCode)
```

This feature is available in the following languages:

Language	Code	Language	Code	Language	Code
Arabic	ar	Croatian	hr	Portuguese - Brazil	pt_BR
Bulgarian	bg	Hungarian	hu	Romanian	ro
Czech	cs	Italian	it	Russian	ru
Danish	da	Hebrew	iw	Slovak	sk
German	de	Japanese	ja	Serbian	sr

Language	Code	Language	Code	Language	Code
Greek	el	Korean	ko	Swedish	sv
English	en	Malay	ms	Thai	th
Spanish	es	Dutch	nl	Turkish	tr
Finnish	fi	Norwegian	no	Simplified Chinese	zh_CN
French	fr	Polish	pl	Traditional Chinese	zh_TW

The function returns an object value with the following properties: `descr`, `parent`, `elements`, `idocMapping`, `vpMapping`, `name`.

Examples:

Using the function:

```
var eventTypeInfo = getEventTypeInfo("com.gttsampleso01.gtt.app.test.ArriveEvent", "zh_CN");
```

Response:

```
{
    "descr": "Arrival Event",
    "parent": {
        "target": "CoreModel.Event"
    },
    "elements": [{
        "name": "stringTest",
        "label": "String Test",
        "type": "string",
        "readable": true,
        "writable": true,
        "translation": {
            "info": {
                "textType": "LABEL",
                "translation": {
                    "zh_CN": "字符串测试"
                }
            },
            "defaultText": "String Test",
            "textType": "LABEL",
            "fieldPath": "label",
            "contextPath": "elements/0",
            "label": {
                "textType": "LABEL",
                "translation": {
                    "zh_CN": "字符串测试"
                }
            }
        },
        "length": 50
    }],
```

```
    "idocMapping": {
        "erpEventCode": "",
        "idoc": "",
        "fieldMapping": []
    },
    "vpMapping": {
        "lbnEventType": "",
        "fieldMapping": []
    },
    "name": "com.gttsampleso01.gtt.app.EventTypeInfoProcess.ArrivalEvent"
}
```

Using a response property:

```
eventTypeInfo.descr
```

getCodeListText

Function `getCodeListText` gets the text of the code from a given code list in a specified language. This function responds in the following ways when dealing with varied situations:

- If there is no text in the specified language, it will default to returning the text in English.
- If the specified code or code list doesn't exist, it will return an empty string.

```
/**
* Get the text of the code.
*
* @param codeType - The name of the code list.
* @param codeValue - The value of the code.
* @param languageCode - A Java locale string.
* @returns String
*/
getCodeListText(codeType, codeValue, languageCode)
```

This feature is available in the following languages:

Language	Code	Language	Code	Language	Code
Arabic	ar	Croatian	hr	Portuguese - Brazil	pt_BR
Bulgarian	bg	Hungarian	hu	Romanian	ro
Czech	cs	Italian	it	Russian	ru
Danish	da	Hebrew	iw	Slovak	sk
German	de	Japanese	ja	Serbian	sr
Greek	el	Korean	ko	Swedish	sv
English	en	Malay	ms	Thai	th
Spanish	es	Dutch	nl	Turkish	tr

Language	Code	Language	Code	Language	Code
Finnish	fi	Norwegian	no	Simplified Chinese	zh_CN
French	fr	Polish	pl	Traditional Chinese	zh_TW

For example:

```
// the variable processStatus is the text in Chinese of Code “AS_PLANNED” of the CodeList “Process
       var processStatus = getCodeListText("ProcessStatus", "AS_PLANNED", "zh_CN");
```

getBPByLBNID

Function `getBPByLBNID` gets the business partner master data via the LBN ID.

```
/**
* Get the business partner master data.
*
* @param LBNID - A string
* @returns ObjectValue of business partner master data
*/
getBPByLBNID(LBNID)
```

The function returns an object value with the following properties:

Response Properties

• `SubDomain` • `Id`: the technical UUID • `LbnId` • `TenantId` • `BpName`	• `bpHeadquaterContactInfo` (including `ContactType`, `Email`, `Phone`, `FirstName`, `LastName`) • `bpContactInfos`(including `ContactType`, `Email`, `Phone`, `FirstName`, `LastName`) • `bpAccountDetails`(including `Status`, `GlobalAccExternalGUID`, `SubAccExternalGUID`, `BpAccId`, `SubscribedPlan`, `BpId`, `BpType`, `SubaccountId`, `SubscribedApp`)

Examples:

Using the function:

```
var bp = getBPByLBNID("LBN#10013165");
```

Response:

```
{
    "SubDomain": "gtt-sample-so-01",
    "TenantId": "20d81d3a-1ff0-49d5-a737-ec98ae7cf587",
    "bpHeadquaterContactInfo": {
        "ContactType": null,
        "Email": "gtt.test@sap.com",
```

```
            "Phone": "13262659870",
            "FirstName": null,
            "LastName": null
        },
        "BpName": "Test Company",
    "Id": "D7964ECC-DE35-4F1E-9090-8D3755B67170",
    "bpContactInfos": [
    {
    "ContactType": "LBN_IT",
    "Email": "lily.lee@abc.com",
    "Phone": "12345678",
    "FirstName": "Lily",
    "LastName": "Lee"
    },
    {
    "ContactType": "LBN_LOGISTICS",
    "Email": "robert.swift@abc.com",
    "Phone": "13312345678",
    "FirstName": "Robert",
    "LastName": "Swift"
    }
    ],
        "bpAccountDetails": [
            {
                "Status": "Active",
                "GlobalAccExternalGUID": "iotDigitalScm",
                "SubAccExternalGUID": "gtt-sample-so-01",
                "BpAccId": "a6e6577b-61cb-4f5c-942c-dc5515c67d4d",
                "SubscribedPlan": "test",
                "BpId": "D7964ECC-DE35-4F1E-9090-8D3755B67170",
                "BpType": "GTT_SOLUTION_OWNER",
                "SubaccountId": "20d81d3a-1ff0-49d5-a737-ec98ae7cf587",
                "SubscribedApp": "lbn_gtt_core_int-app"
            },
            {
                "Status": "Active",
                "GlobalAccExternalGUID": "iotDigitalScm",
                "SubAccExternalGUID": "gtt-sample-so-01",
                "BpAccId": "e7fbb5ad-16a5-4803-90e4-afe8e7ca57ec",
                "SubscribedPlan": "test",
                "BpId": "D7964ECC-DE35-4F1E-9090-8D3755B67170",
                "BpType": "OTHERS",
                "SubaccountId": "20d81d3a-1ff0-49d5-a737-ec98ae7cf587",
                "SubscribedApp": "lbn-dev"
            }
        ],
        "LbnId": "LBN#10013165"
    }
```


Using a response property:

```
// Note that [0] stands for the first business partner account.
      bp.bpAccountDetails[0].status
```

getTimeDifference

Function `getTimeDifference` calculates the time difference between two given timestamp values and returns the result in a specified time unit, such as seconds, minutes, hours, or days.

```
/**
* Get the time difference.
*
* @param timestamp1 - A timestamp value
* @param timestamp2 - A timestamp value
* @param unit - Use “seconds”, “minutes”, “hours”, or “days”. It’s case insensitive.
* @returns Decimal - Rounded to two decimal places using rounding rules. The result is a positive r
*/
getTimeDifference(timestamp1, timestamp2, unit)
```

For example:

```
var timestamp1 = toTimestamp("2022-05-03T12:34:56.000Z");
var timestamp2 = toTimestamp("2022-05-11T17:45:58.000Z");
var unit1 = "days";
var difference1 = getTimeDifference(timestamp1, timestamp2, unit1);
// The timestamp difference1 by unit1 is: -8.22 days
```

generateDirectMailTemplate

Function `generateDirectMailTemplate` defines an instance of the email template object.

```
/**
* Generate a email template.
*
* @returns Email template
*/
reminderService.generateDirectMailTemplate();
directMailTemp.templateId = “name of the email template”;
directMailTemp.locale = “language code”;
directMailTemp.recipients = [“email address list"];
directMailTemp.userdefinedPlaceholders = userDefPlaceholers;
directMailTemp.templateRecipientIgnore = true/false
```

For more information about the properties, see the section Create Mail Object in *Reminder Service for Version 2*.

For example:

```
var directMailTemp = reminderService.generateDirectMailTemplate();
directMailTemp.templateId = “invite”;
directMailTemp.locale = "en";
directMailTemp.recipients = [jane@doe.com, “jack@ryan.com”];
directMailTemp.userdefinedPlaceholders = userDefPlaceholers;
directMailTemp.templateRecipientIgnore = false
```

parseInt